

WEB DEVELOPMENT ASSIGNMENT

Subject: Web Development

Name: Lakshya Agarwal

Roll No.: 2415000885

Course/Branch: B.Tech

TABLE OF CONTENTS

Question 1 — Frontend, Backend, and Full-Stack Development	3
Question 2 — Client-Server Model in Web Architecture (with Diagram)	3
Question 3 — How a Browser Requests and Displays a Web Page.....	4
Question 4 — Tools for a Web Development Environment	5
Question 5 — What is a Web Server? Common Examples	6
Question 6 — Roles: Frontend Developer, Backend Developer, DBA.....	6
Question 7 — Installing and Configuring VS Code (Practical)	7
Question 8 — Static vs Dynamic Websites	9
Question 9 — Web Browsers and Rendering Engines.....	10
Question 10 — Basic Web Architecture Flow (with Diagram)	11
Conclusion.....	11

QUESTION 1

Difference between Frontend, Backend, and Full-Stack Development

Answer:

A website has two main sides, just like a restaurant. One side is what the customer sees and interacts with (the dining area), and the other is what happens behind the scenes to make everything work (the kitchen). Web development is divided in a similar way.

Frontend development (also called client-side development) deals with everything the user sees and interacts with directly in the browser, such as the layout, colours, buttons, forms, and menus. It is built using HTML, CSS, and JavaScript, along with frameworks like React or Angular. For example, when you open Instagram and see the feed, like buttons, and stories, that visual part is built by frontend developers.

Backend development (also called server-side development) deals with the logic, database, and server that work behind the scenes. It handles things like storing user data, processing login requests, and sending back the right information. It is built using languages such as Node.js, Python, Java, or PHP, along with databases like MySQL or MongoDB. For example, when you log in to Instagram, the backend checks your username and password against the database and decides whether to let you in.

Full-stack development means working on both the frontend and backend. A full-stack developer can build the complete website or application from start to finish — the part the user sees and the part that runs behind the scenes. For example, a single developer building a complete food-ordering website (the menu page, the cart, and the payment processing) from scratch is doing full-stack development.

Aspect	Frontend	Backend	Full-Stack
Focus	User interface and experience	Server logic, database, security	Both frontend and backend
Runs on	User's browser	Web server	Both browser and server
Common tools	HTML, CSS, JavaScript, React	Node.js, Python, Java, PHP	Combination of both sets
Real-world example	Designing the Instagram feed layout	Processing an Instagram login request	Building a complete food-ordering app

QUESTION 2

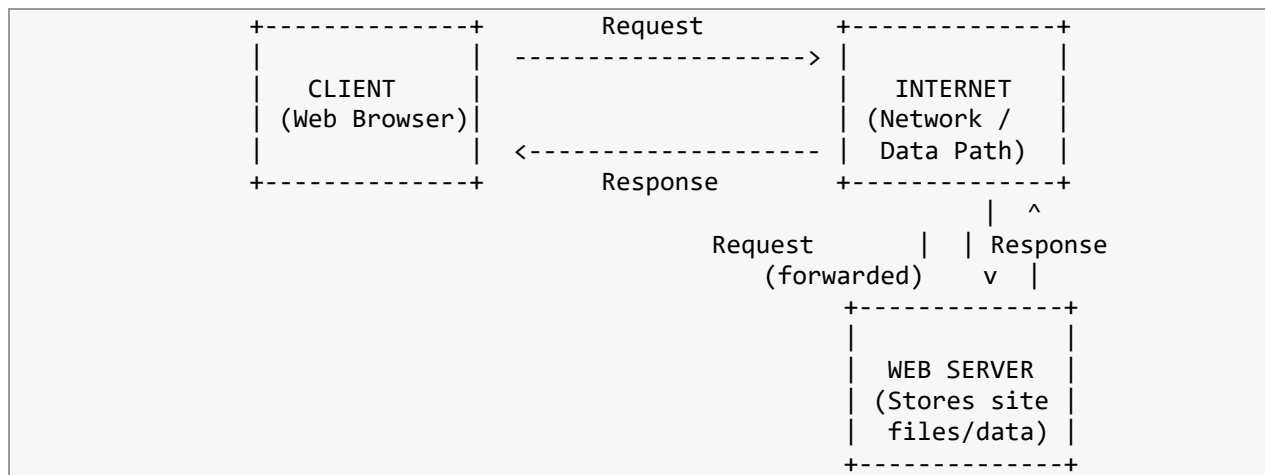
Client-Server Model in Web Architecture

Answer:

The client-server model is the basic structure on which the entire web works. In simple words, one computer (the client) asks for something, and another computer (the server) gives it back. It is similar to how a customer orders food at a counter and the shop prepares and hands over the order.

The client is usually a web browser (like Chrome or Firefox) running on the user's device. The server is a powerful computer that stores websites and data, and stays ready to respond to requests at any time. When a user wants to visit a website, the browser (client) sends a request over the Internet, and the server sends back the requested web page as a response, which the browser then displays.

Diagram: Client-Server Model



Explanation of the flow:

- Client/Browser: The user opens a browser and types a website address or clicks a link.
- Request: The browser sends a request through the Internet asking for the web page.
- Internet: Acts as the path/network that carries the request from the client to the correct server.
- Web Server: Receives the request, finds or prepares the requested page, and sends it back.
- Response: The server's reply (HTML, CSS, images, etc.) travels back through the Internet.
- Client/Browser: The browser receives the response and displays the web page to the user.

QUESTION 3

How a Browser Requests and Displays a Web Page

Answer:

When we type a website address in the browser and press Enter, a series of steps happen very quickly behind the scenes before the page appears on our screen. These steps are explained below in simple order.

1. URL Entry: The user types a web address (URL), such as `www.example.com`, into the browser's address bar.
2. DNS Lookup: The browser contacts a DNS (Domain Name System) server, which works like a phonebook for the Internet, to convert the readable domain name into the server's actual numeric IP address.
3. Establishing Connection: The browser opens a connection to the web server at that IP address, usually using the HTTP or HTTPS protocol.
4. Sending the Request: The browser sends an HTTP request asking the server for the specific web page.
5. Server Processing: The web server receives the request, locates or generates the requested page (HTML file, along with linked CSS and JavaScript files), and prepares a response.

6. Sending the Response: The server sends the requested files back to the browser as an HTTP response.
7. Rendering: The browser's rendering engine reads the HTML, CSS, and JavaScript files and converts them into the visual web page — arranging text, images, colours, and layout.
8. Display: The fully rendered page is displayed on the user's screen, ready to be viewed and interacted with.

This entire process usually takes less than a second, which is why web pages seem to load almost instantly even though many steps are happening in the background.

QUESTION 4

Tools Required to Set Up a Web Development Environment

Answer:

Before starting web development, a few basic tools need to be set up on the computer. Each tool has a specific purpose, explained below.

Tool	Purpose
Code Editor (e.g., Visual Studio Code)	Used to write and edit HTML, CSS, JavaScript, and other code. It provides helpful features like syntax highlighting, auto-complete, and error detection.
Web Browser (e.g., Chrome, Firefox)	Used to open and test the website, and to check how it looks and behaves for the end user.
Browser Developer Tools	Built into browsers; used to inspect HTML/CSS, debug JavaScript, and check network requests while testing a site.
Version Control System (e.g., Git)	Used to track changes in code over time and to collaborate with other developers without overwriting each other's work.
Hosting Platform for Version Control (e.g., GitHub)	Used to store code online, share projects, and collaborate with a team.
Local/Live Server (e.g., VS Code Live Server extension)	Used to preview the website locally in real time as changes are made, without needing to upload it online.
Package Manager (e.g., npm)	Used to install and manage external libraries, frameworks, and tools needed for a project.
Backend Runtime/Language (e.g., Node.js, Python, PHP)	Used to write server-side logic that handles data, user requests, and business rules.
Database Software (e.g., MySQL, MongoDB)	Used to store, organise, and retrieve data such as user accounts, products, or messages.
Design/Prototyping Tool (e.g., Figma)	Used to plan and design the look of a website before actually coding it.

QUESTION 5

What is a Web Server? Common Examples

Answer:

In simple words, a web server is like a librarian who keeps all the books (website files) safely stored and hands over the correct one whenever someone asks for it.

Technically, a web server is a computer (along with software running on it) that stores website files — such as HTML, CSS, JavaScript, images, and data — and delivers them to a client's browser whenever a request is made over the Internet using the HTTP or HTTPS protocol. A web server stays running continuously so that the website is available to visitors at any time.

Commonly Used Web Servers:

Web Server	Note
Apache HTTP Server	One of the oldest and most widely used open-source web servers, known for flexibility and wide community support.
Nginx	Known for high performance and handling many requests at once; often used for large, high-traffic websites.
Microsoft IIS (Internet Information Services)	A web server built for Windows Server environments, commonly used with Microsoft technologies like ASP.NET.
LiteSpeed	A high-speed commercial web server often used as a faster alternative to Apache.
Node.js (as a server)	Not a traditional web server by itself, but JavaScript code run on Node.js is commonly used to create custom web servers.

QUESTION 6

Roles of Frontend Developer, Backend Developer, and Database Administrator

Answer:

A web development project usually involves different people focusing on different parts of the system. Their roles are explained below.

Role	Main Responsibilities
Frontend Developer	Designs and builds everything the user sees and interacts with: page layout, styling, buttons, forms, and navigation. Ensures the website looks good and works smoothly on different screen sizes and devices. Uses HTML, CSS, JavaScript, and frontend frameworks.
Backend Developer	Builds the server-side logic that powers the website: handling requests, processing data, managing user authentication, and connecting to the database. Ensures the application runs correctly, securely, and efficiently behind the scenes.
Database Administrator (DBA)	Designs, organises, and maintains the database that stores the website's data. Responsible for data security, backups, performance tuning, and making sure data can be accessed quickly and reliably by the backend.

In smaller projects, one person (a full-stack developer) may take on more than one of these roles, while in larger companies, these are usually handled by separate specialists working together as a team.

QUESTION 7

Install and Configure VS Code for HTML, CSS, and JavaScript Development

1. System Requirements / Pre-requisites

- Operating System: Windows 10 or later, macOS 10.15 or later, or a modern Linux distribution.
- RAM: At least 4 GB (8 GB or more recommended for smooth performance).
- Storage: At least 500 MB of free disk space for VS Code (more for extensions and projects).
- A stable Internet connection (needed to download VS Code and extensions).
- A modern web browser installed (Chrome, Firefox, or Edge) to preview the project.

2. Steps to Download and Install Visual Studio Code

1. Open a web browser and go to the official website: <https://code.visualstudio.com>
2. Click the "Download" button. The site will automatically detect the operating system (Windows/macOS/Linux) and offer the correct installer.
3. Once the installer file is downloaded, open/run it.
4. On Windows: Accept the licence agreement, and it is recommended to tick options such as "Add to PATH" and "Create a desktop icon" during setup, then click Next/Install.
5. On macOS: Drag the downloaded VS Code icon into the Applications folder.
6. On Linux: Install the downloaded .deb or .rpm package using the package manager, or the Software Centre.
7. Once installed, open Visual Studio Code from the Start Menu, Applications folder, or desktop shortcut.

3. Steps to Configure VS Code for HTML, CSS, and JavaScript

1. Open VS Code and create a new folder for the project (File → Open Folder).
2. Inside this folder, create the basic files needed: index.html, style.css, and script.js.
3. Go to the Extensions panel (the square icon on the left sidebar, or press Ctrl+Shift+X) to install helpful extensions (listed below).
4. Enable "Auto Save" from File → Auto Save, so changes are saved automatically while testing.
5. Set up Prettier (after installing it) as the default formatter: open Settings (Ctrl+,), search for "default formatter", and choose Prettier. This keeps code neat and consistently formatted.

4. Recommended Extensions

Extension	Purpose
Live Server	Runs the project on a local server and automatically refreshes the browser whenever a file is saved.
Prettier – Code Formatter	Automatically formats HTML, CSS, and JavaScript code neatly and consistently.
ESLint	Checks JavaScript code for errors and enforces good coding practices.
HTML CSS Support	Provides smart suggestions and autocomplete for CSS class/ID names inside HTML files.
Auto Rename Tag	Automatically renames the matching closing HTML tag when the opening tag is edited.
Path Intellisense	Auto-completes file paths when linking files (like images or scripts) in code.

5. Simple Test Project

Below is a simple test project with three files to confirm that the setup is working correctly.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Test Project</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, Web Development!</h1>
  <p>This is a simple test page to check my VS Code setup.</p>
  <button onclick="showMessage()">Click Me</button>

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  text-align: center;
  padding: 50px;
}

h1 {
  color: #2c3e50;
}

button {
  padding: 10px 20px;
  background-color: #2980b9;
  color: white;
  border: none;
```

```
border-radius: 5px;  
cursor: pointer;  
}
```

script.js

```
function showMessage() {  
  alert("VS Code setup is working correctly!");  
}
```

6. Running the Project Using Live Server

1. Make sure the Live Server extension is installed (as listed above).
2. Open the index.html file in VS Code.
3. Right-click anywhere inside the file and select "Open with Live Server" (or click the "Go Live" button in the bottom-right corner of the VS Code window).
4. The default browser will automatically open and display the web page, usually at an address like `http://127.0.0.1:5500/index.html`
5. Any changes made and saved in the HTML, CSS, or JavaScript files will automatically refresh the browser, allowing instant preview.

7. Screenshot Placeholder

[INSERT SCREENSHOT HERE — VS CODE HTML/CSS/JAVASCRIPT SETUP]

What the screenshot should show:

- VS Code open with the project folder visible in the sidebar (showing index.html, style.css, and script.js).
- The index.html file open in the editor with its code visible.
- The Extensions panel or installed extensions list showing Live Server and Prettier installed.
- Ideally, the browser window (opened via Live Server) visible alongside VS Code, showing the test page ("Hello, Web Development!" with the button) running live.

QUESTION 8

Static Websites vs Dynamic Websites

Answer:

A static website is one whose content stays the same for every visitor and does not change unless a developer manually edits the code. It is built using only HTML and CSS (sometimes with a little JavaScript for simple effects), and pages are shown exactly as they are stored on the server. For example, a simple personal portfolio website or a one-page business "About Us" website that displays the same information to every visitor is a static website.

A dynamic website is one whose content can change based on user interaction, input, time, or data stored in a database. It uses a backend (server-side programming) along with a database to generate pages on

the fly. For example, an online shopping website like Amazon is dynamic — it shows different products, prices, and recommendations depending on the user, their search, and their location.

Aspect	Static Website	Dynamic Website
Content	Fixed, same for every user	Changes based on user/data/time
Technology used	HTML, CSS (basic JS)	HTML, CSS, JS + backend + database
Speed	Usually faster to load	Can be slower due to processing
Example	A simple personal portfolio site	An online shopping site like Amazon

QUESTION 9

Five Web Browsers and Their Rendering Engines

Answer:

What is a Rendering Engine?

In simple words, a rendering engine is like a translator and painter combined — it reads the HTML, CSS, and JavaScript code of a web page and "paints" it on the screen as the visual page we actually see and use.

Technically, a rendering engine (also called a browser engine or layout engine) is the software component inside a browser that parses HTML and CSS, builds the page structure (DOM and CSSOM), calculates the layout of every element, and draws the final page on the screen. It also works closely with the browser's JavaScript engine to handle interactivity.

Five Web Browsers and Their Rendering Engines:

Browser	Rendering Engine
Google Chrome	Blink
Mozilla Firefox	Gecko
Microsoft Edge	Blink
Apple Safari	WebKit
Opera	Blink

How These Rendering Engines Differ:

- **Blink (Chrome, Edge, Opera):** Developed by Google (originally forked from WebKit in 2013). It is fast, widely used, and supports the latest web standards quickly. Because Chrome, Edge, and Opera all use Blink, websites tend to look and behave consistently across these three browsers.
- **Gecko (Firefox):** Developed by Mozilla. It is an independent engine (not based on Blink or WebKit) known for strong privacy features and good standards support, though it is used by a smaller share of browsers.

- WebKit (Safari): Developed by Apple, and it is the engine from which Blink was originally forked. It is optimised for Apple devices and is known for good performance and battery efficiency on Mac and iPhone.

In short, even though these browsers look different on the outside, the engine underneath decides how quickly and accurately a web page is displayed, and small differences between engines can sometimes cause a website to look slightly different or behave differently across browsers — which is why developers test their websites on multiple browsers.

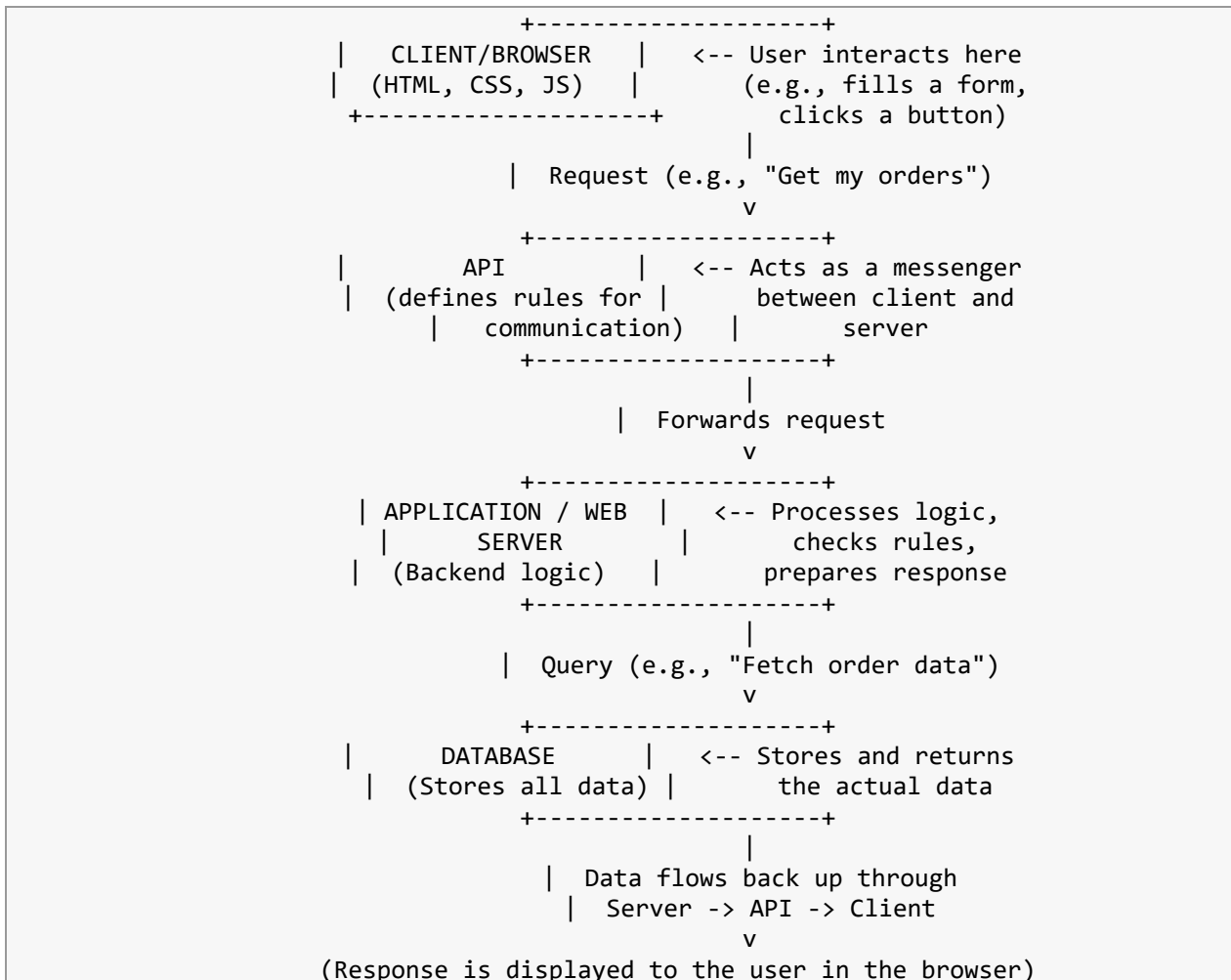
QUESTION 10

Basic Web Architecture Flow: Client, Server, Database, and APIs

Answer:

Modern websites and applications usually follow a layered flow of communication between four main parts: the client, the API, the application/web server, and the database. Each layer has a specific job, and data passes through them in order.

Diagram: Basic Web Architecture Flow



Explanation of Each Part:

- Client/Browser: The starting point, where the user interacts with the interface — for example, clicking "View Orders" on a shopping website.
- API (Application Programming Interface): Acts as a messenger or middle layer with a defined set of rules, allowing the client and server to communicate without needing to know each other's internal workings.
- Application/Web Server: Receives the request forwarded by the API, applies the business logic (such as checking if the user is logged in), and decides what data is needed.
- Database: Stores all the actual data (such as user accounts, orders, and products) and returns the requested data back to the server when queried.
- Response Flow: Once the database sends the data back, it travels back up through the server and the API, and is finally displayed to the user in the browser in a readable format.

CONCLUSION

This assignment covered the fundamental building blocks of web development, starting from the difference between frontend, backend, and full-stack roles, to how the client-server model and web architecture work together to deliver a web page to the user. It also covered practical aspects such as setting up a development environment and configuring VS Code for actual coding, along with conceptual topics such as static versus dynamic websites and how different browsers render web pages using different engines.

Overall, this assignment helped in understanding not just the theory behind how websites work, but also the practical tools and workflow used by web developers in real projects. This foundation is essential before moving on to building actual websites and web applications.